UOR

Faculty of Technology, University of Ruhuna

Faculty of Technology Academic Management System

Software Requirements Specification (SRS)

Course: ICT2132 — Object Oriented Programming Practicum

Implementation: Java desktop GUI application

Database: MySQL

Document version: 1.0 | **Date:** 8 September 2026

Prepared as an academic system design specification

Table of Contents

1. Introduction
2. Overall Description
3. System Overview
4. Functional Requirements
5. User Roles and Permissions
6. Detailed Module Requirements
7. Business Rules
8. Non-Functional Requirements
9. Object-Oriented Design Requirements
10. System Architecture
11. UML Diagrams
12. DFD Diagrams
13. Activity Diagrams
14. Sequence Diagrams
15. State Diagrams
16. Database Design
17. ER Diagram
18. Database Table Specifications
19. Validation Rules
20. Error and Exception Handling
21. GUI Requirements
22. Reports
23. Requirement Traceability Matrix
24. Assumptions and Limitations
25. Future Enhancements
26. Conclusion

Appendix A — Sample Data

Appendix B — Glossary

1. Introduction

1.1 Purpose

This SRS defines the functional, data, interface, quality, and object-oriented design requirements for a Java and MySQL desktop application that manages academic information for the Faculty of Technology. It is the baseline for analysis, design, implementation, testing, and demonstration in ICT2132.

1.2 Scope

The system manages role-based profiles, courses, materials, attendance, medical records, configurable assessments and marks, eligibility, grades, SGPA, CGPA, notices, timetables, and reports. It supports Admin, Lecturer, Technical Officer, and Undergraduate users. It is not an online student portal or a learning-management system.

1.3 Product Perspective

The product is a standalone, single-institution desktop information system using a Java GUI and a central MySQL database. It replaces fragmented spreadsheets and manual calculations with controlled data entry, consistent calculations, and read-only views appropriate to each role.

1.4 Definitions and Acronyms

Term	Meaning
CA	Continuous Assessment mark for an enrolled course; it is used for eligibility.
SGPA / CGPA	Semester / cumulative Grade Point Average, calculated as credit-weighted grade points.
Batch-missed	A student who follows a course with a later or different batch because the original scheduled attempt was missed.
Repeat student	A student repeating a course attempt under the institution's approved academic process.
DAO	Data Access Object: Java class that isolates SQL/database operations.

1.5 References

[R1] University Grants Commission, Sri Lanka, Commission Circular No. 12/2024: Guidance of the Grading System of Degree Programmes offered under Technology Stream, 11 October 2024. The circular states that the amended guidance applies to Technology-degree students enrolled from the 2022/2023 batch onward. [R2] MySQL 8 Reference Manual. [R3] Java Platform, Standard Edition documentation.

2. Overall Description

2.1 Product Perspective

The system has one source of truth for academic records. The GUI sends validated requests to controllers/services; services apply academic rules; DAOs use transactions to read and update MySQL records.

2.2 Product Functions

- Authenticate users and show a dashboard based on their role.
- Maintain profiles, courses, components, assessments, notices, and timetables.
- Record theory/practical attendance and approved medical evidence.
- Capture configurable assessments, calculate CA, eligibility, final marks, grades, SGPA, and CGPA.
- Provide batch, course, and student reports without granting unapproved edit access.

2.3 User Classes

User class	Typical knowledge	Primary responsibility
Admin	Trained staff user	Master data, accounts, notices, timetables
Lecturer	Academic staff	Materials, assessment marks, academic views
Technical Officer	Laboratory/support staff	Attendance and medical records
Undergraduate	Student	Own permitted profile and academic information

2.4 Operating Environment

Java SE 17+ desktop client (Swing or JavaFX), JDBC driver, MySQL 8+, Windows/Linux laboratory or office PCs, and a local/institutional network connection to MySQL.

2.5 Constraints

The application must use Java, MySQL, a GUI, and demonstrable OOP concepts. Every mark is out of 100. It must operate offline from the public Internet, except where the institution later chooses hosted database infrastructure.

2.6 Assumptions

> **Assumption A-01.** The faculty supplies authorised course offerings, enrolments, timetable data, and final local approval for academic policies. This mini project implements those approved records; it does not replace Senate or examination-board decisions.

2.7 Dependencies

MySQL availability, JDBC configuration, Java runtime, accurate timetable setup, and authorised staff data entry are required for correct operation.

3. System Overview

Inputs

Profiles, course and component data, assessment definitions, marks, session attendance, medical details, notices, timetable entries.

Outputs

Role dashboards, attendance percentages, medical status, eligibility, grade/result views, SGPA/CGPA, and printed/exportable reports.

> **Calculation scope:** Grade and GPA calculations use the grading scheme stated in Section 7. If a Faculty regulation differs from the published guidance, the administrator must configure the approved scheme and retain the approval reference; the system must not silently change historical results.

4. Functional Requirements

1. **FR-001** The system shall authenticate an active user by username and password and load only the functions assigned to the user role.
2. **FR-002** Admin shall create, read, update, deactivate, and maintain Admin, Lecturer, Technical Officer, and Undergraduate profiles.
3. **FR-003** Admin shall create and maintain departments, batches, courses, course offerings, components, notices, and timetables.
4. **FR-004** Lecturer and Technical Officer shall update their own profile except username and password; undergraduates shall update only contact details and profile picture.
5. **FR-005** Lecturer shall add and modify course materials for assigned course offerings.
6. **FR-006** The system shall allow configured assessment types per course offering, rather than assuming one common evaluation pattern.
7. **FR-007** Lecturer shall enter and revise marks out of 100 for authorised assessments and view them by student, batch, subject, and CA.
8. **FR-008** Technical Officer shall create and maintain timetable-linked theory and practical attendance sessions and records for current-semester modules.
9. **FR-009** The system shall show theory-only, practical-only, and combined attendance where both components exist.
10. **FR-010** Technical Officer shall create and maintain medical records; authorised users shall view their permitted medical records.
11. **FR-011** The system shall calculate attendance, CA, attendance eligibility, CA eligibility, and final examination eligibility.
12. **FR-012** The system shall calculate final marks, grades, grade points, SGPA, and CGPA and show supporting courses, credits, marks, and grades.
13. **FR-013** Users shall view notices and authorised timetable views; Technical Officers view only their department timetable.
14. **FR-014** The system shall produce the reports listed in Section 22 and keep an audit record of sensitive changes.

5. User Roles and Permissions

5.1 Major-module permission matrix

Module / role	Admin				Lecturer				Technical Officer				Undergraduate			
	C	R	U	D	C	R	U	D	C	R	U	D	C	R	U	D
User & profiles	√	√	√	√*	—	own/ student	own	—	—	own	own	—	—	own	limited	—
Courses / offerings	√	√	√	√*	—	assigned	—	—	—	department	—	—	—	enrolled	—	—
Materials	—	—	—	—	√	√	√	√	—	—	—	—	—	enrolled	—	—
Attendance / medical	—	√	—	—	—	√	—	—	√	√	√	√	—	own	—	—
Assessments / marks	—	√	—	—	√	√	√	authorised	—	—	—	—	—	own results	—	—
Notices / timetables	√	√	√	√	—	√	—	—	—	department	—	—	—	√	—	—

C=Create, R=Read, U=Update, D=Delete. *Delete means soft-delete/deactivate where dependent academic records exist; hard deletion is prohibited. “Authorised” means assigned course/department and permitted current semester scope.

5.2 Access-control requirements

FR-015 Each request shall be checked by the service layer, not only by hiding GUI buttons. FR-016 A user shall never read or change a record outside the role and data scope in this matrix.

6. Detailed Module Requirements

6.1 Authentication & Authorisation

Purpose/actors: secure role-based access for all users. **Pre/post:** active account exists → authenticated session and role dashboard. **Inputs:** username, password. **Processing/output:** validate, compare password hash, log event, return dashboard or error. **Validation/errors:** blank/invalid credentials, locked/inactive account, unauthorised action.

6.2 User, Student, Lecturer & Technical Officer Management

Purpose/actors: Admin maintains role profiles; each role maintains permitted own fields. **Pre/post:** authorised actor → valid linked user/role profile is saved or deactivated. **Inputs:** identity, contact, department/batch/status details. **Processing/output:** check unique username/student ID, validate role scope, save profile; updated profile/list. **Errors:** duplicate identity, invalid email/phone, missing role link.

6.3 Course Management

Purpose/actors: Admin defines courses, offerings, credits, theory/practical components and enrolments. **Pre/post:** department exists → current-semester offering is available to authorised data entry. **Inputs:** code, title, credits, components, semester, batch. **Processing/output:** validate code/credits, prevent duplicate offering, save; course catalogue and offering view. **Errors:** invalid credit/component, duplicate course code, enrolment conflict.

6.4 Course Material Management

Purpose/actors: Lecturer publishes resources for assigned offerings. **Pre/post:** assignment exists → material metadata/path is available to enrolled students. **Inputs:** title, description, controlled file path/link. **Processing/output:** verify course ownership and file metadata; material list. **Errors:** unsupported file reference, missing title, unassigned lecturer.

6.5 Attendance Management

Purpose/actors: Technical Officer records timetable-linked current-semester attendance; lecturer/student read authorised views. **Pre/post:** offering, component, and timetable slot exist → one attendance record per student/session. **Inputs:** session date, component, enrolled students, presence. **Processing/output:** prevent duplicates, calculate counts/percentages; batch/student/component reports. **Errors:** session outside semester, absent enrolment, duplicate record, invalid component.

6.6 Medical Management

Purpose/actors: Technical Officer maintains submitted medical evidence and approval. **Pre/post:** student/enrolment exists → medical status is retained and used by eligibility rule. **Inputs:** student, course/period, date, reason, evidence reference, status. **Processing/output:** validate date/range, store pending/approved/rejected; medical view and eligibility input. **Errors:** future date, missing reason, overlapping duplicate record.

6.7 Assessment & Marks Management

Purpose/actors: Lecturer configures assessment components and records marks. **Pre/post:** lecturer assignment and offering exist → marks are saved against an assessment/enrolment. **Inputs:** assessment type/ name, weight, CA flag, mark 0–100. **Processing/output:** validate total configured weights and mark range; mark, student, batch, subject, and CA views. **Errors:** duplicate assessment/mark, weight total invalid, unauthorised course, mark outside range.

6.8 Eligibility, Grade & SGPA/CGPA Management

Purpose/actors: system calculates academic outcomes; lecturer/student read authorised views. **Pre/post:** required attendance and marks exist → derived eligibility/result snapshot is displayed. **Inputs:** attendance records, medical approval, assessment marks, credits. **Processing/output:** apply Sections 7 and 16 rules; eligibility, grades, SGPA/CGPA with supporting courses. **Errors:** incomplete configured assessment, unavailable grade scheme, inconsistent credits.

6.9 Notice, Timetable, Reports, Profile & Dashboard

Purpose/actors: Admin publishes notices/timetables; all roles see scoped dashboards and reports. **Pre/post:** authorised record exists → targeted users receive readable current information. **Inputs:** title/body/audience/date or timetable slot/location. **Processing/output:** validate time collisions and audience; filtered views/reports. **Errors:** expired/blank notice, timetable overlap, invalid target audience.

7. Business Rules

ID	Rule
BR-001	Only an active user with a verified password hash may create an authenticated session.
BR-002	Role permission and data scope must be enforced for every create, read, update, delete, and report operation.
BR-003	Each student may have one active enrolment per course offering; repeat/batch-missed attempts are separate enrolment records with an attempt number and status.
BR-004	Every assessment belongs to one course offering, has a configurable type, and records marks from 0 to 100.
BR-005	CA eligibility is true only when the calculated CA mark is at least 40.00%.
BR-006	Attendance percentage for a component = (eligible attended sessions ÷ scheduled sessions) × 100. A theory/practical view uses only the matching component. Combined percentage = (eligible attended theory + practical sessions) ÷ (scheduled theory + practical sessions) × 100.
BR-007	The standard scheduled count is 15 theory sessions and 15 practical sessions for a component offered in the current semester. A one-credit theory/practical component represents two contact hours per session; credits define course load, not a multiplier for the session percentage.
BR-008	**Assumption:** attendance eligibility requires at least 80.00% in every required component. For a course with both components, the combined value is displayed, but eligibility requires each required component to meet 80% unless the Faculty formally approves a different policy.
BR-009	**Assumption:** an approved medical record may convert eligible absence sessions within its approved date range into medical-excused sessions for the attendance calculation; those sessions are removed from the denominator, not counted as present. Rejected/pending medical records do not change attendance.
BR-010	Final examination eligibility is true only if CA eligibility and attendance eligibility are both true. The system shall display the failed condition(s).
BR-011	Final mark is the sum of each configured assessment mark × its configured weight / 100. Weights for a published assessment plan must total 100%.
BR-012	The grade scheme and grade points follow the Technology-stream guidance of UGC Circular 12/2024 for applicable cohorts. The approved scheme is versioned so historic results remain reproducible.
BR-013	SGPA = Σ(course credit × grade point) ÷ Σ(course credits counted in that semester). CGPA = Σ(course credit × grade point across counted completed attempts) ÷ Σ(counted course credits). Values display to two decimals.
BR-014	**Assumption:** for a repeated course, the approved result-selection rule must be set by the Faculty; the mini-project default is to include the latest completed attempt in CGPA and retain all attempts for audit.
BR-015	Passwords are never stored in plaintext. Profile image storage uses a controlled file path/UUID in the database; the GUI validates type and access location.

7.1 Verified grading reference

The official circular is cited in [R1]. The table below represents the published Technology-stream scale used by this proposed system. **Assumption:** a programme-specific approved exception, if any, must be supplied by the Faculty before deployment and configured as a new scheme version.

Mark range	Grade	Grade point
85–100	A+	4.00
80–84	A	4.00
75–79	A−	3.70
70–74	B+	3.30
65–69	B	3.00
60–64	B−	2.70
55–59	C+	2.30
50–54	C	2.00
45–49	C−	1.70
40–44	D+	1.30
35–39	D	1.00
0–34	E	0.00

8. Non-Functional Requirements

ID	Requirement
NFR-001	Normal list, login, and single-result operations shall respond within 3 seconds on the specified local network under expected class-size data.
NFR-002	Passwords shall be salted and hashed using a modern Java-supported password-hashing algorithm; database credentials shall not be hard-coded in source control.
NFR-003	GUI labels, validation messages, and workflow order shall be understandable to second-year users; critical actions require confirmation.
NFR-004	All multi-table changes shall use database transactions; failed writes shall roll back and display a meaningful message.
NFR-005	Layered Java classes, meaningful names, unit-testable services, and DAO interfaces shall support maintenance.
NFR-006	The schema shall support additional courses, offerings, batches, assessments, and users without redesign.
NFR-007	Foreign keys, unique constraints, checks, and application validation shall protect data integrity.
NFR-008	Availability depends on the local Java client and MySQL service; the system shall recover gracefully from a temporary connection loss.
NFR-009	Errors shall not expose SQL, password, or stack-trace details to ordinary users; technical detail is logged securely.
NFR-010	Authentication and authorisation shall be checked at login and before protected service operations.
NFR-011	Daily logical backup and periodic restore testing are required operational procedures; database backup files must be access-controlled.
NFR-012	Log login attempts and create/update/delete events for marks, attendance, medical approvals, profiles, and master data with actor and timestamp.
NFR-013	The application shall run on supported Windows/Linux Java environments and MySQL 8+ through JDBC.
NFR-014	Keyboard navigation, readable labels, sufficient contrast, visible focus, and non-colour-only status indicators shall be used where practical in the desktop GUI.

9. Object-Oriented Design Requirements

Concept	Required use in this system
Classes & objects	Domain objects such as Course, Enrollment, Assessment, AttendanceSession, MedicalRecord, Notice, and Grade represent real academic concepts.
Encapsulation	Private fields with validated getters/setters; for example, `Marks.setScore()` rejects values outside 0–100 and `Assessment.setWeight()` rejects invalid percentages.
Inheritance & abstraction	Abstract `User` holds shared identity/authentication properties. Admin, Lecturer, TechnicalOfficer, and Undergraduate extend it and implement role-specific behaviour.
Interfaces	`Repository<T>`, `Authenticatable`, and `EligibilityPolicy` reduce coupling and permit substitution in tests.
Polymorphism / overriding	`User.getDashboardPermissions()` and `EligibilityPolicy.evaluate()` are invoked through common types but execute role/policy-specific implementations.
Access modifiers	Private data fields; protected shared user behaviour; public validated service APIs. DAO SQL helpers remain package-private/private where possible.
Exception handling	Custom ValidationException, AuthorizationException, NotFoundException, and DataAccessException are caught at controller boundaries and translated to safe GUI messages.
Database handling	DAO classes use JDBC prepared statements, result mapping, transactions, and try-with-resources; services must not build SQL in the GUI.
GUI	Views collect input and render models only; controllers coordinate views and services so business rules remain reusable/testable.

10. System Architecture

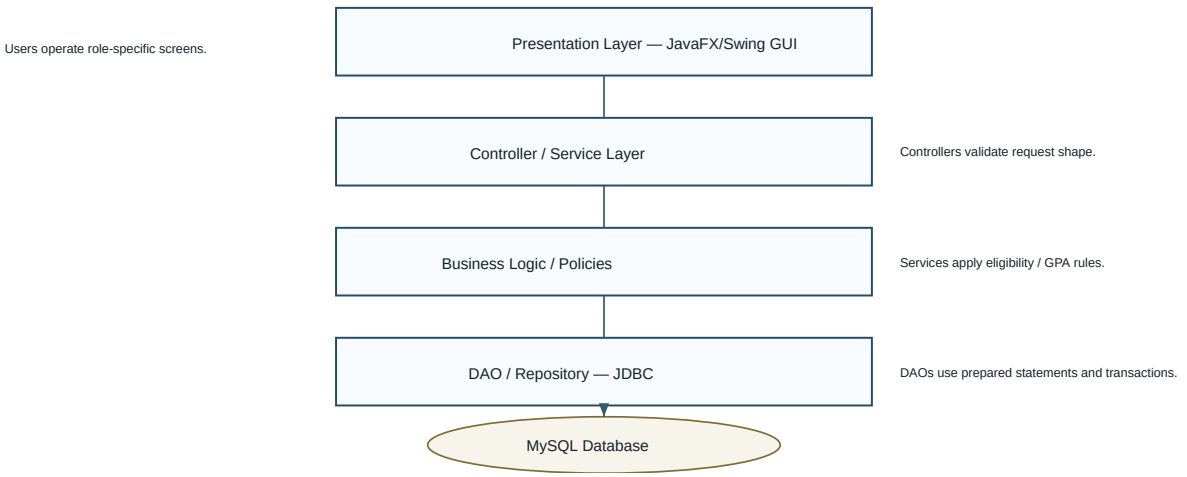


Figure 1. Layered architecture. Dependencies flow down; results/errors return to the GUI.

10.1 Authentication, connection and transaction flow

1. GUI submits credentials to AuthController; AuthService obtains the active user through UserDAO and compares a password hash.
2. Service creates a role-aware session object and writes an audit event; no plaintext password is retained.
3. DAO obtains a JDBC connection from a configured connection provider. Prepared statements prevent SQL injection.
4. For an atomic operation (for example, publish assessment and marks), the service starts a transaction, calls DAOs, commits only after all steps succeed, otherwise rolls back.
5. Input validation occurs in GUI for immediate feedback and again in service/domain objects for protection. Exceptions are logged and converted to user-safe messages.

11. UML Diagrams

11.1 Use Case Diagram

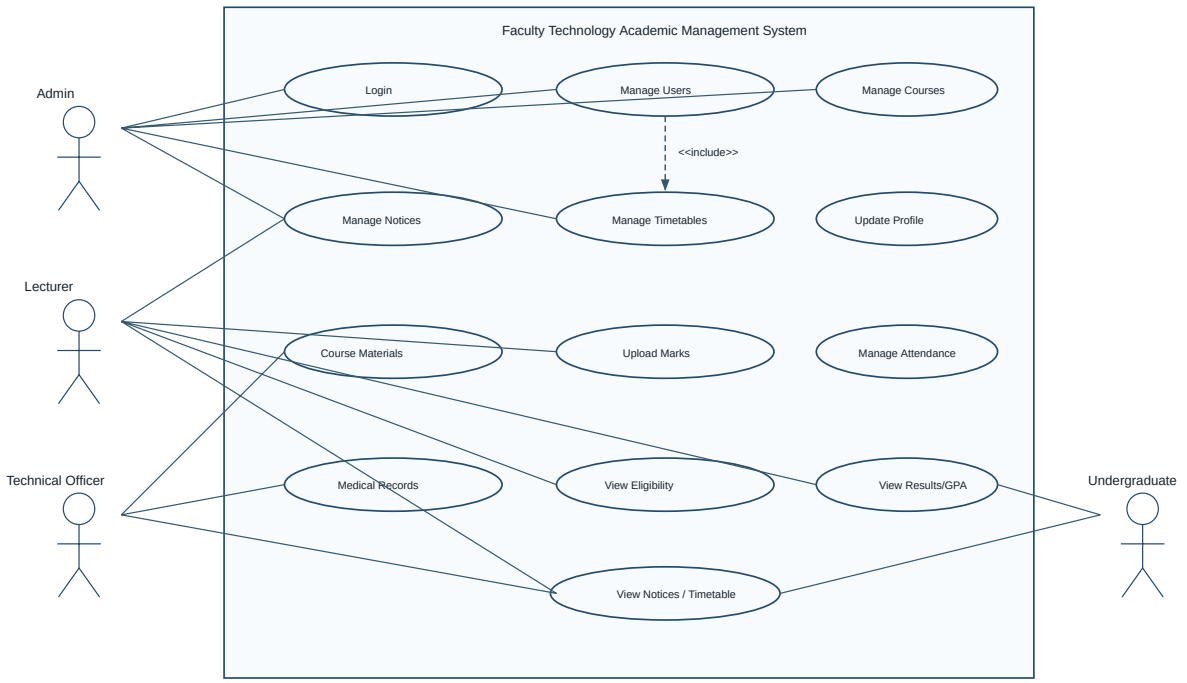


Figure 2. Major use cases. Dashed inclusion indicates that user management requires an authenticated session; other associations show permitted role use.

11.2 Detailed Class Diagram

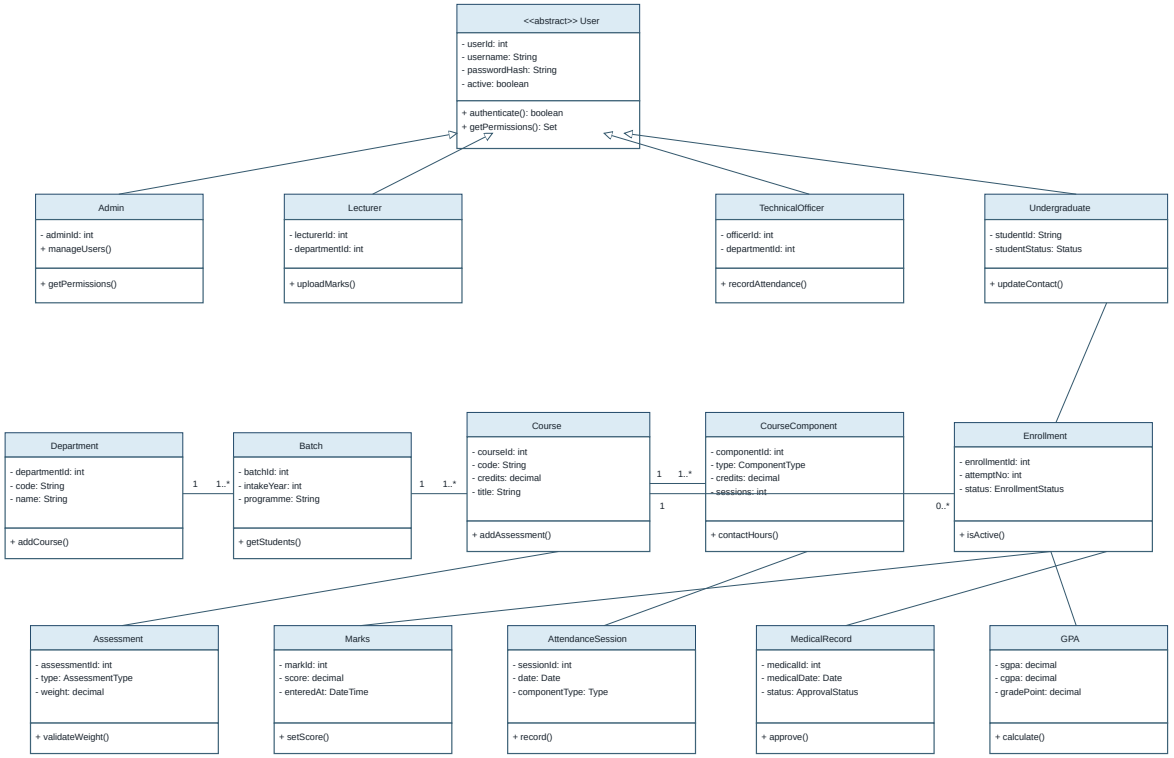


Figure 3. Domain class diagram. Course components and assessments are composed by a course offering; enrolment resolves student-to-course many-to-many associations.

12. DFD Diagrams

12.1 Level 0 Context Diagram

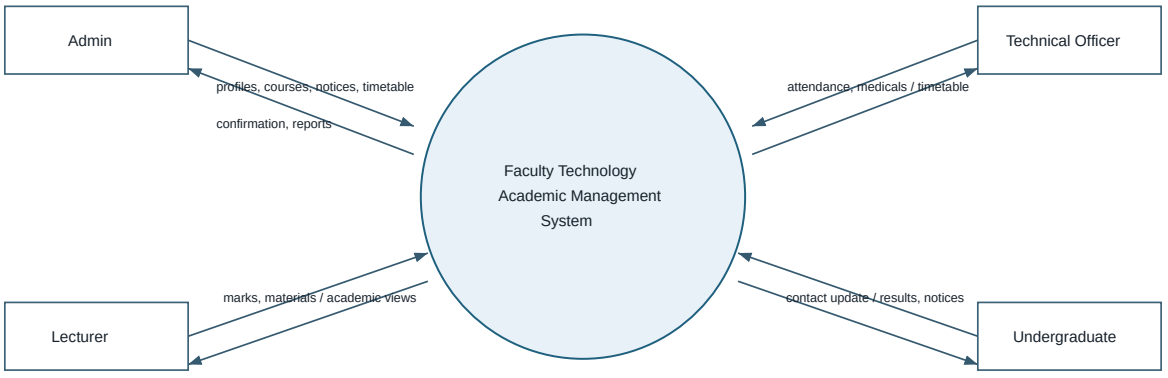


Figure 4. Context DFD: principal external entities and major information flows.

12.2 Level 1 DFD

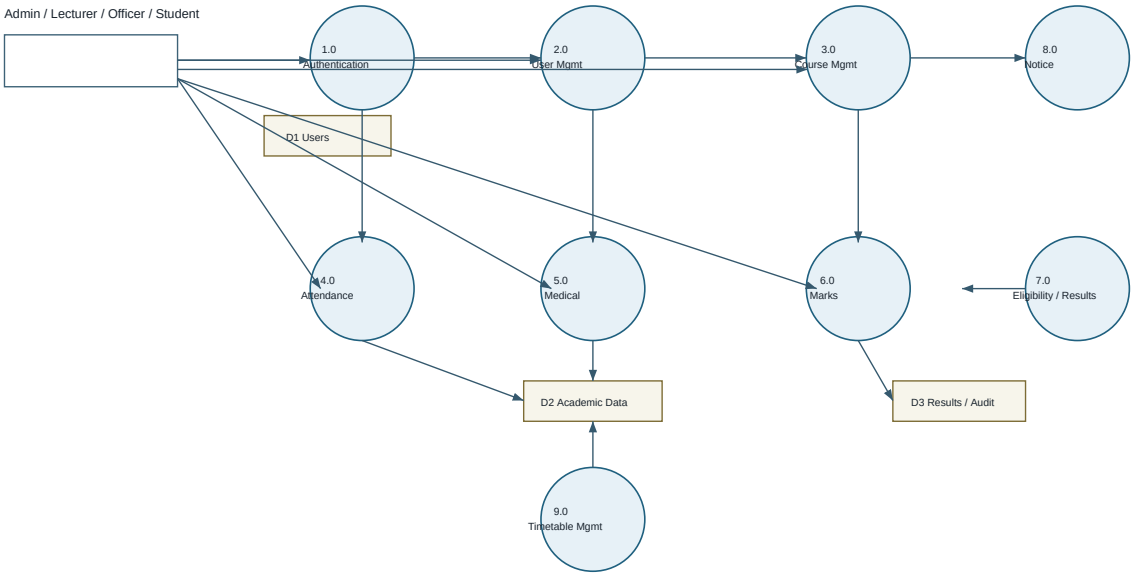


Figure 5. Level 1 DFD. D1 contains accounts; D2 course, attendance and medical information; D3 derived results and audit events.

13. Activity Diagrams

Each workflow begins at the filled circle, performs user/system actions, evaluates a decision, follows an alternate path where needed, and ends at the double circle.

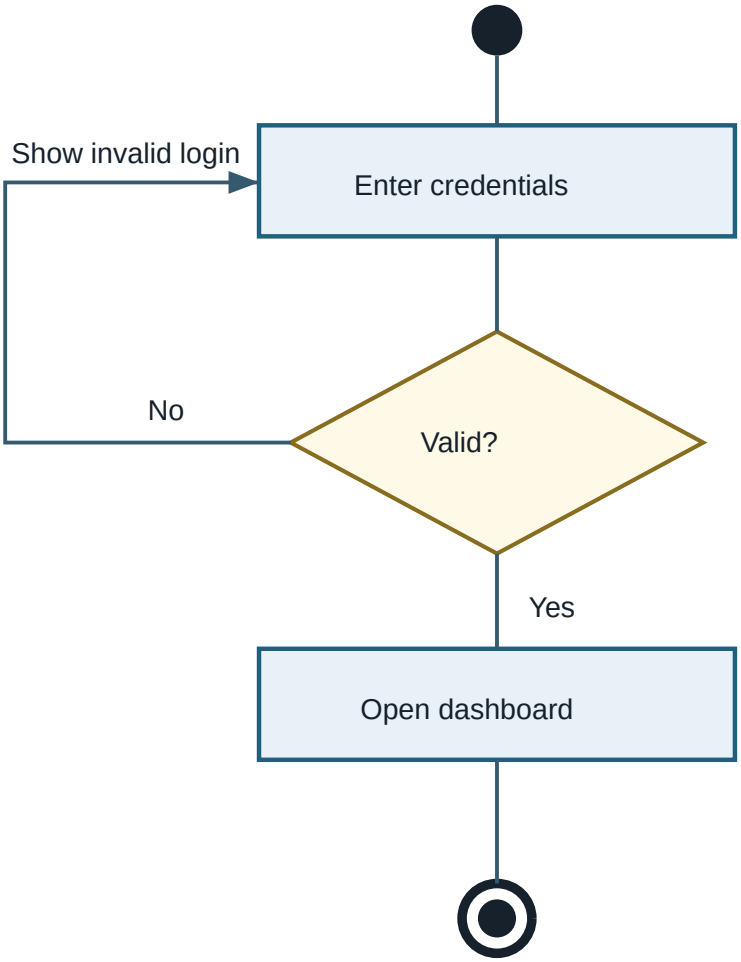


Figure 6. Login activity — alternative invalid-login path returns to credential entry.

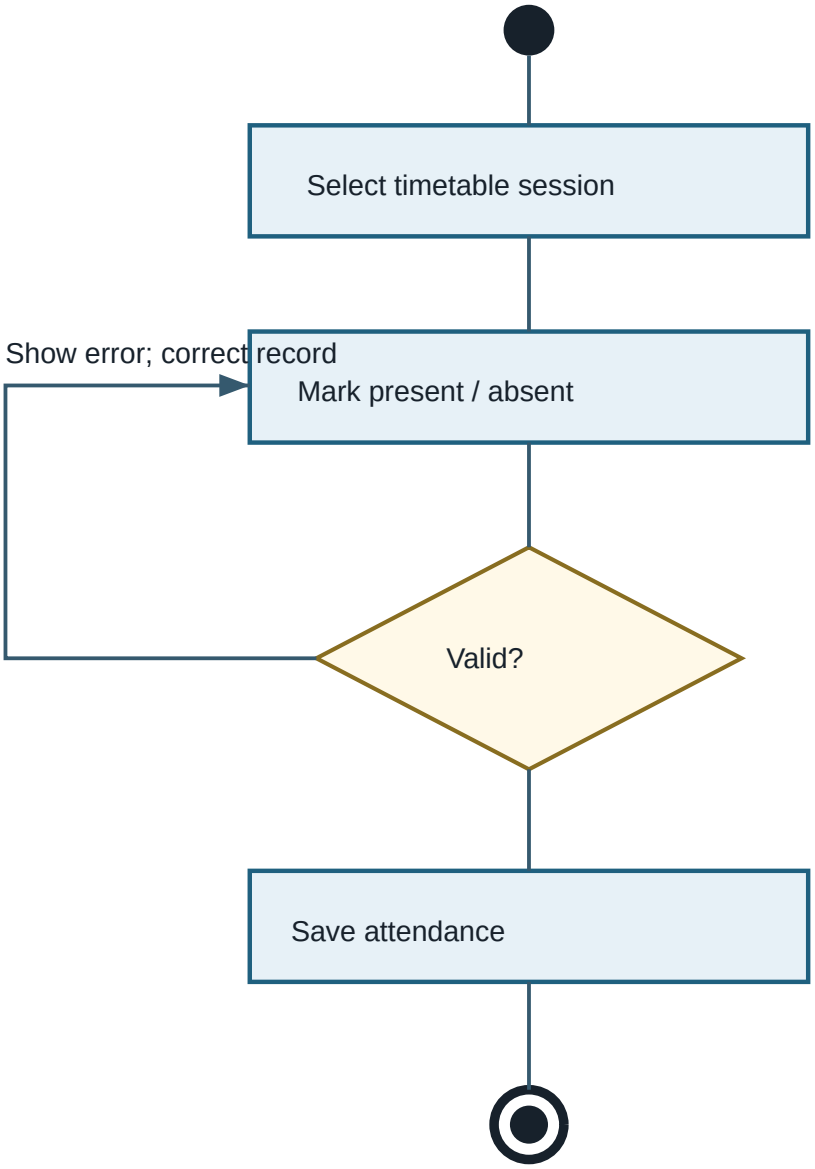


Figure 7. Attendance activity — verifies timetable, component and enrolment.

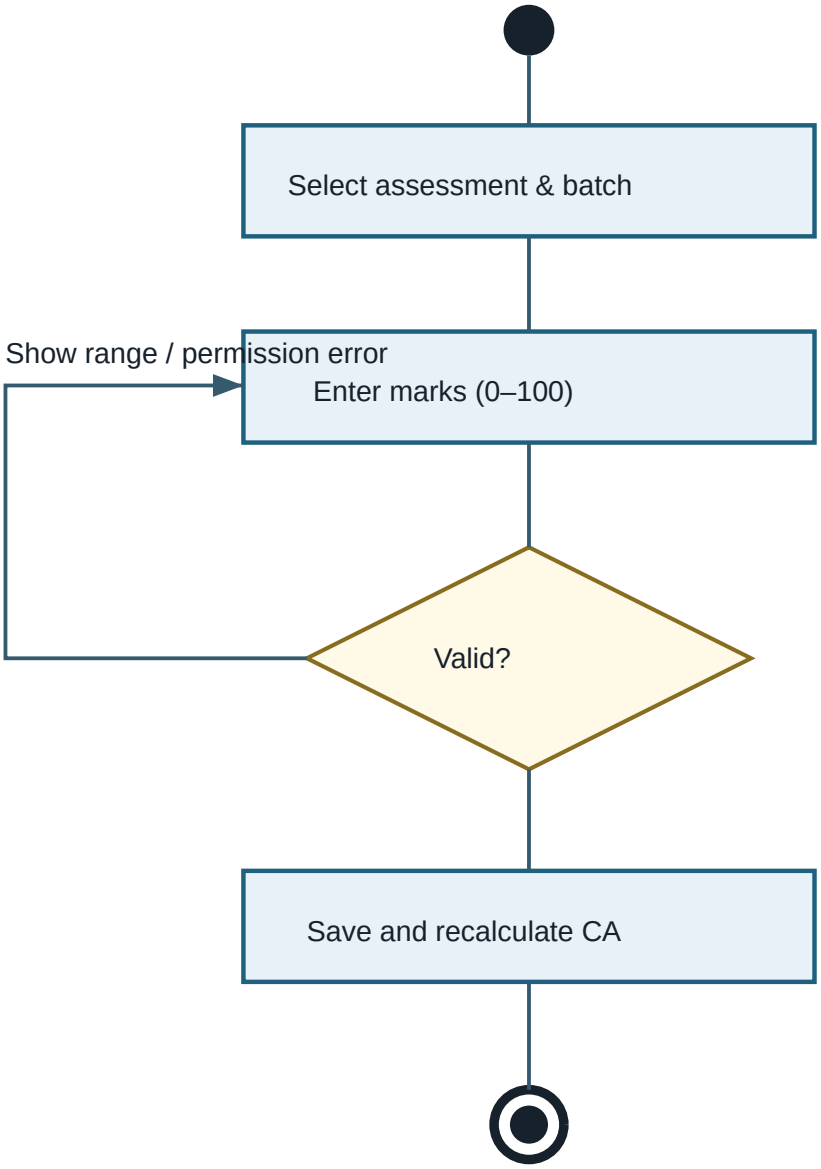


Figure 8. Upload marks activity — validation precedes persistence and CA update.

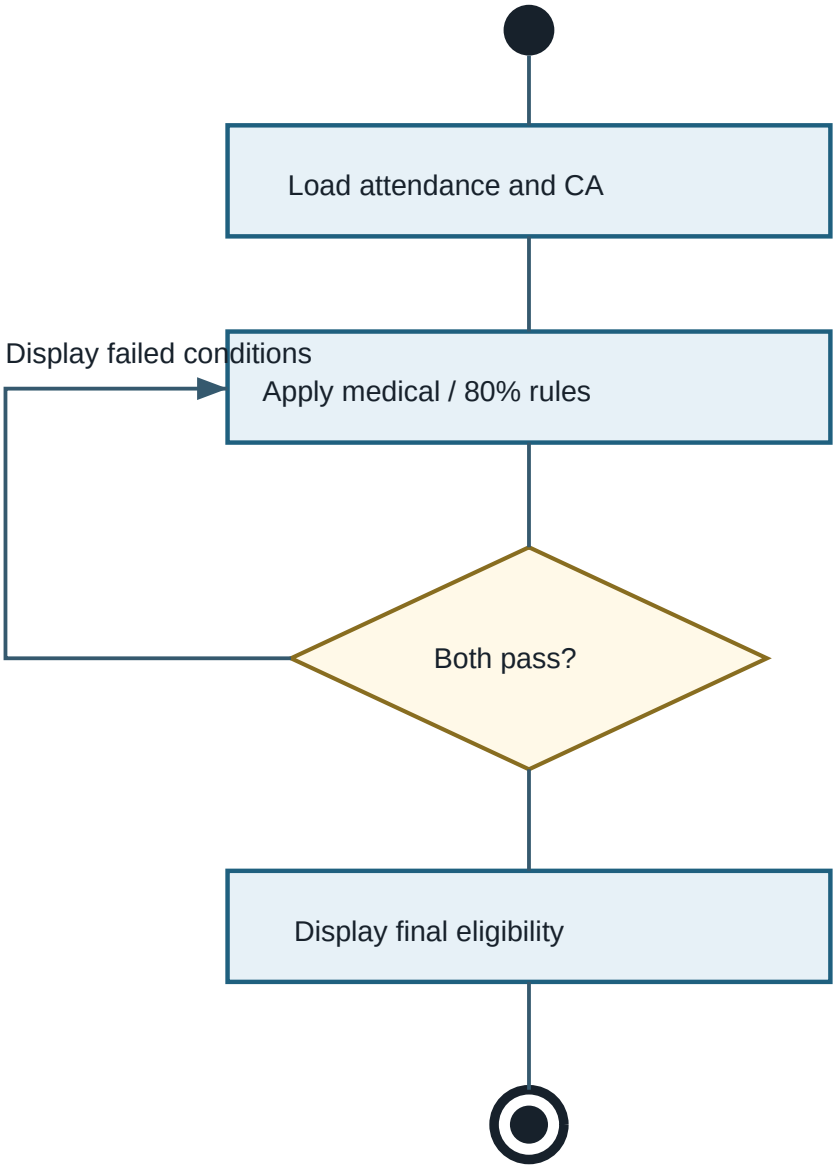


Figure 9. Eligibility activity — shows a reason when attendance or CA fails.

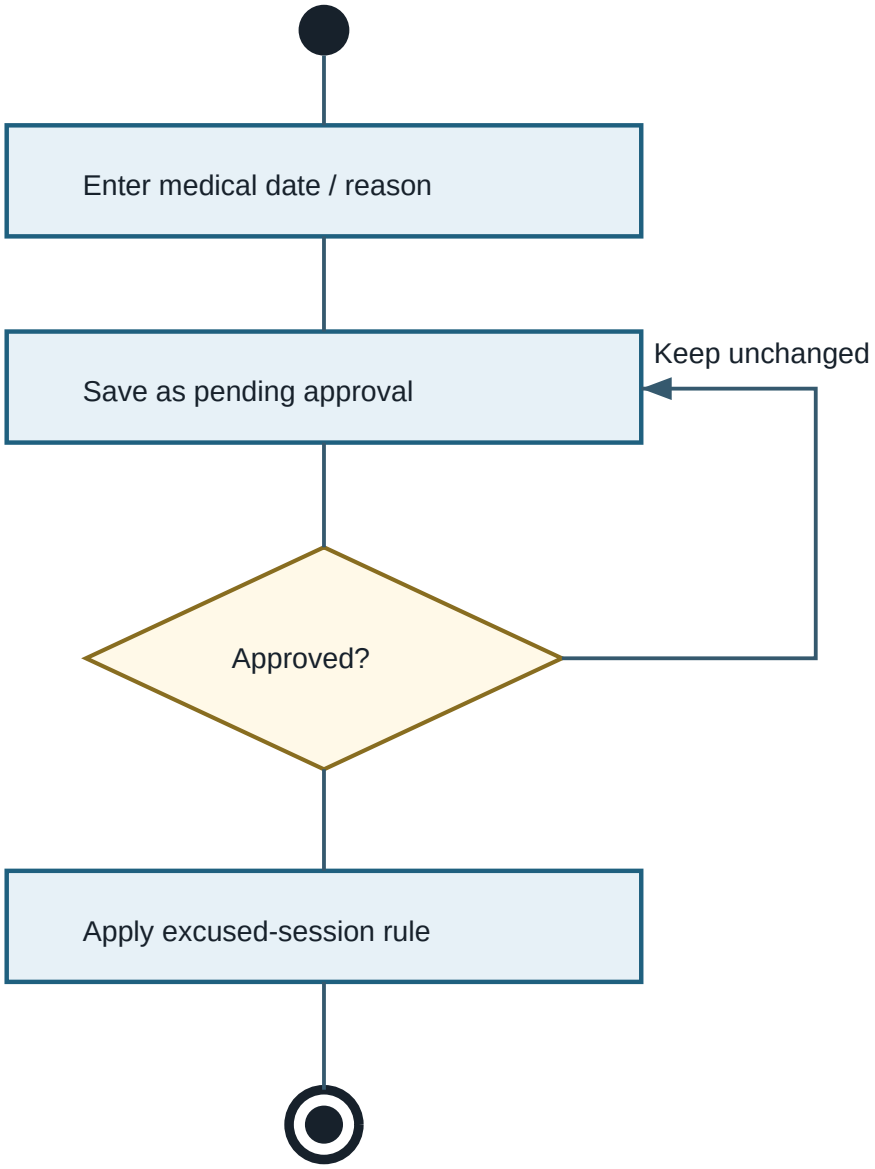


Figure 10. Medical activity — pending/rejected evidence does not change attendance.

13.1 Additional activity specifications

Workflow	Start → actions → decision/alternative → end
Calculate GPA	Start → select student/semester → load completed counted enrolments → verify credits/grades available → calculate weighted GPA or display incomplete-result reason → show SGPA/CGPA → end.
Manage Course	Start → Admin enters course/offering/components → validate unique code, credits and components → save or show validation error → show catalogue → end.
Publish Notice	Start → Admin enters title/body/audience/date → validate required fields and dates → publish or correct error → show filtered notice → end.

14. Sequence Diagrams

All sequences preserve the required path: GUI → Controller/Service → Business Logic → DAO → Database, with return values propagated back to the GUI.

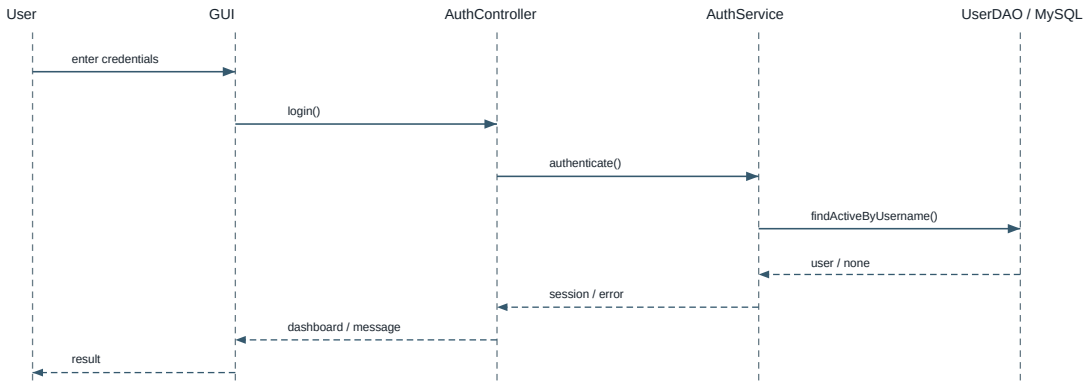


Figure 11. Login sequence.

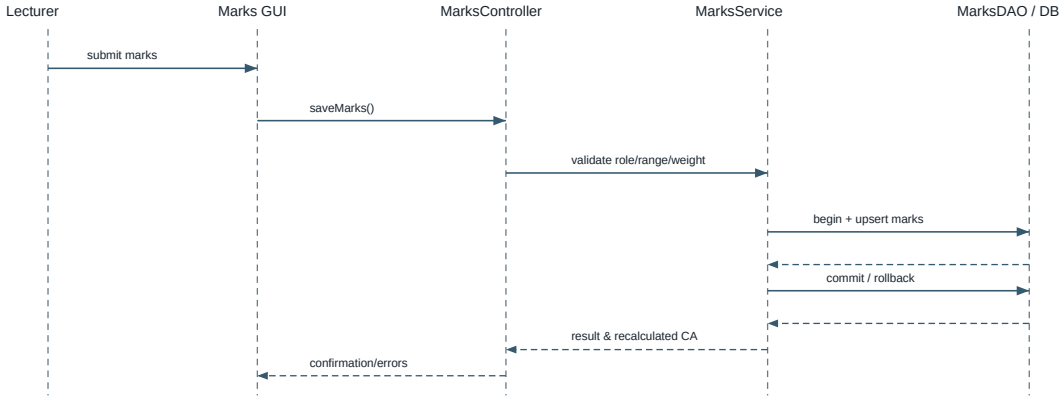


Figure 12. Lecturer marks-upload sequence.

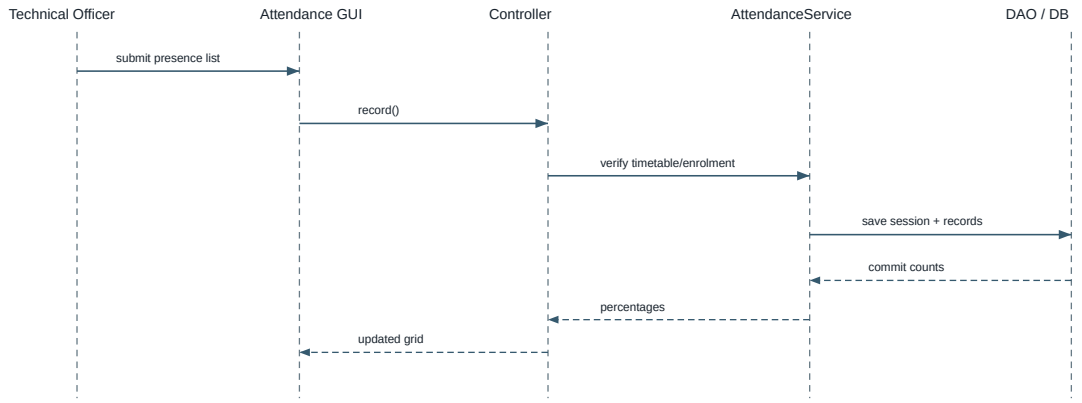


Figure 13. Technical Officer attendance sequence.

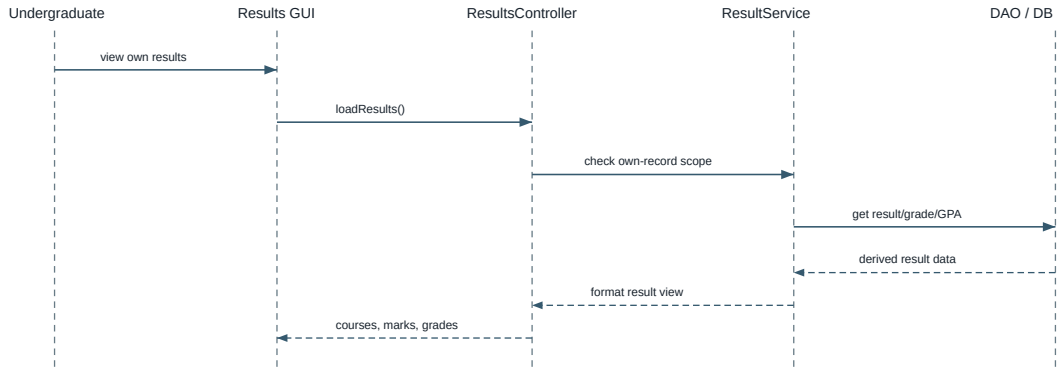


Figure 14. Student results-view sequence.

14.1 Eligibility-checking sequence

Actor: Lecturer or system result process. **Messages:** Eligibility GUI requests evaluation → Controller calls EligibilityService → service loads attendance, medical approvals, CA and configuration through DAOs → service applies BR-005 to BR-010 → DAO may persist an auditable result snapshot → GUI displays eligible/not eligible and failed conditions. This is the same layered interaction pattern shown in Figures 11–14.

15. State Diagrams

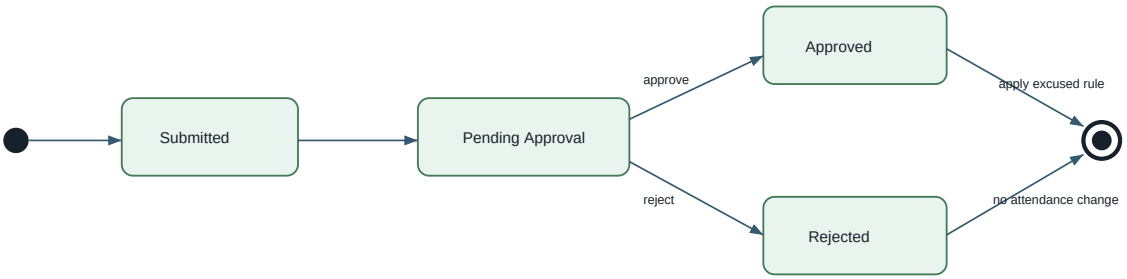


Figure 15. Medical Record state diagram. The submitted record is placed in Pending Approval until an authorised decision is recorded.

Useful secondary state: Assessment is Draft → Published → Marks Open → Locked. A published assessment cannot be changed if marks already exist without a controlled authorised revision/audit event.

16. Database Design

16.1 Design approach

The schema is normalised to practical third normal form: a person account is separated from role-specific data; student-to-offering, course-to-lecturer, and assessment-to-mark relationships use junction/child tables; derived GPA is calculated from results rather than duplicated as a permanent source of truth. Profile pictures use a controlled relative file path/UUID rather than BLOB to keep routine database backups and queries lightweight.

16.2 Keys, indexes and referential actions

Category	Specification
Primary keys	Every table has a surrogate INT/BIGINT auto-increment primary key, except `users` may use BIGINT. Business identifiers (username, course code, student number) are alternate unique keys.
Foreign keys	All relationship columns in the field specification are foreign keys. Use **RESTRICT** for academic parent records with history; use **CASCADE** only for true dependent children such as attendance_records when an unposted session is removed.
Unique constraints	users.username; undergraduates.student_no; departments.code; courses.course_code; course_offerings(course_id,semester_id,batch_id); course_enrollments(student_id,offering_id,attempt_no); assessments(offering_id,name); marks(assessment_id,enrollment_id); attendance_records(session_id,enrollment_id).
Important indexes	FK indexes; course_enrollments(student_id, status); attendance_sessions(offering_id, session_date); medical_records(student_id, approval_status); marks(enrollment_id); notices(publish_date, expiry_date); timetable_entries(timetable_id, day_of_week, start_time).
Validation constraints	credits > 0; mark 0–100; weight 0–100; attendance presence boolean; status values via CHECK/ENUM; start_time < end_time; approved date range valid. MySQL 8 CHECK constraints are used; service validation remains mandatory.

16.3 Relationship table

Parent table	Child table	Relationship	Foreign key	Cardinality	Explanation
users	admins/lecturers/ technical_officers/ undergraduates	role profile	user_id	1 : 0..1 per role	Shared credentials, role-specific fields.
departments	batches, courses, lecturers, technical_officers	owns/employs	department_id	1 : many	Academic department scope.
batches	undergraduates, course_offerings	contains/ offered to	batch_id	1 : many	Supports normal, repeat and batch-missed enrolments through enrolment status.
courses	course_components, course_offerings	defines/offers	course_id	1 : many	Course definition is reused across semesters.
course_offerings	enrollments, assessments, sessions, materials	academic instance	offering_id	1 : many	Current semester scoped operations.
undergraduates + offerings	course_enrollments	junction	student_id, offering_id	many : many	Captures attempt/status.
assessments + enrollments	marks	junction/score	assessment_id, enrollment_id	many : many	One score per enrolled student/assessment.
attendance_sessions + enrollments	attendance_records	junction/status	session_id, enrollment_id	many : many	Presence per student/ session.
undergraduates	medical_records	has	student_id	1 : many	Medical approval supports attendance eligibility rule.

17. ER Diagram

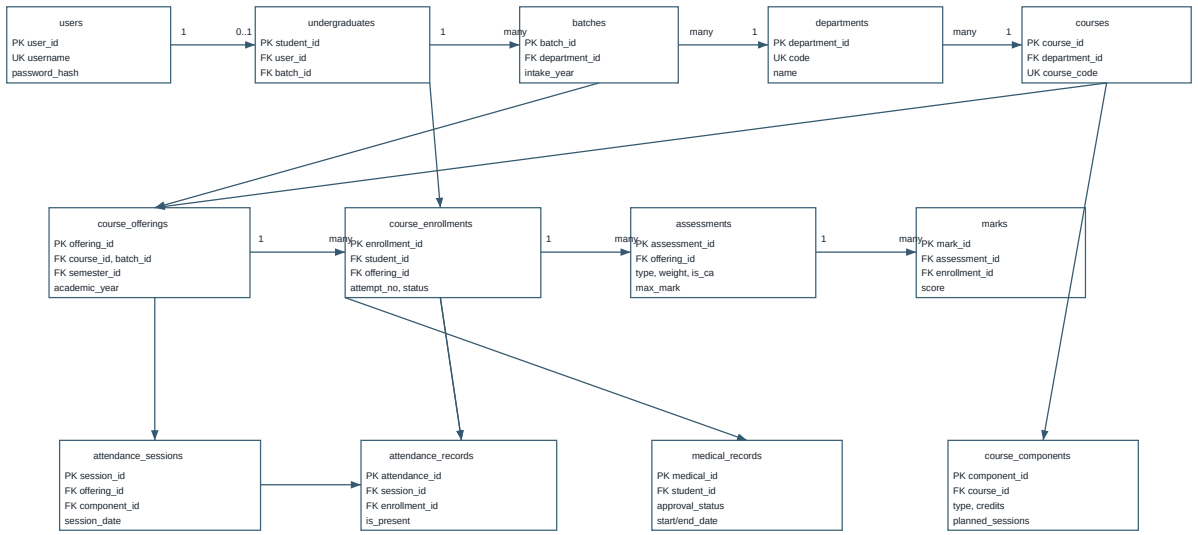


Figure 16. MySQL ER diagram. Lines identify valid FK-to-PK relationships; enrolments, marks and attendance records resolve many-to-many relationships.

18. Database Table Specifications

All table names are lowercase snake_case. **Key**: PK=primary key, FK=foreign key, UK=unique key. Data types are MySQL 8 types. **DB-001:** Every listed FK must reference a valid parent key. **DB-002:** Required foreign-key indexes, checks and unique rules from Section 16 shall be created.

Table	Field	Data type	Size	Constraints	Key	Description
users	user_id	BIGINT	—	NOT NULL AUTO_INCREMENT	PK	Account identifier
	username	VARCHAR	50	NOT NULL	UK	Login name
	password_hash	VARCHAR	255	NOT NULL	—	Salted password hash
	role	ENUM	—	NOT NULL	INDEX	ADMIN/LECTURER/ TECHNICAL_OFFICER/ UNDERGRADUATE
	email	VARCHAR	120	NOT NULL	UK	Contact email
	is_active	BOOLEAN	—	NOT NULL DEFAULT TRUE	—	Soft access status
	created_at	DATETIME	—	NOT NULL	—	Audit time
admins	admin_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Admin identifier
	user_id	BIGINT	—	NOT NULL	FK, UK	→ users.user_id ON DELETE RESTRICT
	full_name	VARCHAR	120	NOT NULL	—	Name
	phone	VARCHAR	20	NULL	—	Contact
departments	department_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Department identifier
	code	VARCHAR	15	NOT NULL	UK	Department code
	name	VARCHAR	120	NOT NULL	—	Department name
	office_location	VARCHAR	80	NULL	—	Office
	is_active	BOOLEAN	—	DEFAULT TRUE	—	Status
batches	batch_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Batch identifier
	department_id	INT	—	NOT NULL	FK	→ departments, RESTRICT
	intake_year	YEAR	—	NOT NULL	—	Intake year
	programme	VARCHAR	100	NOT NULL	—	Programme title
	batch_code	VARCHAR	20	NOT NULL	UK	Human-readable batch
undergraduates	student_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Student identifier
	user_id	BIGINT	—	NOT NULL	FK, UK	→ users, RESTRICT
	batch_id	INT	—	NOT NULL	FK, INDEX	→ batches, RESTRICT
	student_no	VARCHAR	25	NOT NULL	UK	Institution student number
	full_name	VARCHAR	120	NOT NULL	—	Name
	profile_picture_path	VARCHAR	255	NULL	—	Controlled file path/UUID
lecturers	lecturer_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Lecturer identifier
	user_id	BIGINT	—	NOT NULL	FK, UK	→ users
	department_id	INT	—	NOT NULL	FK, INDEX	→ departments
	full_name	VARCHAR	120	NOT NULL	—	Name

	phone	VARCHAR	20	NULL	—	Contact
	designation	VARCHAR	80	NULL	—	Role title
technical_officers	officer_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Officer identifier
	user_id	BIGINT	—	NOT NULL	FK, UK	→ users
	department_id	INT	—	NOT NULL	FK, INDEX	→ departments
	full_name	VARCHAR	120	NOT NULL	—	Name
	phone	VARCHAR	20	NULL	—	Contact
	designation	VARCHAR	80	NULL	—	Role title
courses	course_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Course identifier
	department_id	INT	—	NOT NULL	FK, INDEX	→ departments
	course_code	VARCHAR	20	NOT NULL	UK	Course code
	title	VARCHAR	180	NOT NULL	—	Course title
	total_credits	DECIMAL	3,1	NOT NULL CHECK >0	—	Total credits
	is_active	BOOLEAN	—	DEFAULT TRUE	—	Status
course_components	component_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Component identifier
	course_id	INT	—	NOT NULL	FK, INDEX	→ courses
	component_type	ENUM	—	NOT NULL	—	THEORY/PRACTICAL
	credits	DECIMAL	3,1	NOT NULL CHECK >0	—	Component credits
	planned_sessions	INT	—	NOT NULL DEFAULT 15 CHECK >0	—	Planned sessions

Table	Field	Data type	Size	Constraints	Key	Description
semesters	semester_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Semester identifier
	academic_year	VARCHAR	9	NOT NULL	UK pair	e.g., 2026/2027
	semester_no	TINYINT	—	NOT NULL CHECK IN (1,2)	UK pair	Semester number
	start_date	DATE	—	NOT NULL	—	Start
	end_date	DATE	—	NOT NULL CHECK >start_date	—	End
course_offerings	offering_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Course instance
	course_id	INT	—	NOT NULL	FK	→ courses
	batch_id	INT	—	NOT NULL	FK	→ batches
	semester_id	INT	—	NOT NULL	FK	→ semesters
	coordinator_id	INT	—	NULL	FK	→ lecturers
	status	ENUM	—	DEFAULT 'OPEN'	UK tuple	OPEN/CLOSED
course_enrollments	enrollment_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Enrollment identifier
	student_id	INT	—	NOT NULL	FK, INDEX	→ undergraduates
	offering_id	INT	—	NOT NULL	FK, INDEX	→ course_offerings
	attempt_no	TINYINT	—	NOT NULL DEFAULT 1	UK tuple	Attempt number
	student_status	ENUM	—	NOT NULL	—	NORMAL/REPEAT/BATCH_MISSED
	enrollment_status	ENUM	—	NOT NULL	—	ACTIVE/COMPLETED/WITHDRAWN
assessments	assessment_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Assessment identifier
	offering_id	INT	—	NOT NULL	FK, INDEX	→ offerings
	name	VARCHAR	100	NOT NULL	UK pair	Assessment name
	assessment_type	ENUM	—	NOT NULL	—	ASSIGNMENT/QUIZ/MID/PRACTICAL/CA/FINAL/OTHER
	weight_percent	DECIMAL	5,2	NOT NULL CHECK 0–100	—	Contribution
	is_ca_component	BOOLEAN	—	NOT NULL	—	Counts toward CA
marks	mark_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Mark identifier
	assessment_id	INT	—	NOT NULL	FK	→ assessments
	enrollment_id	INT	—	NOT NULL	FK	→ enrollments
	score	DECIMAL	5,2	NOT NULL CHECK 0–100	UK pair	Raw mark out of 100
	entered_by	INT	—	NOT NULL	FK	→ lecturers
attendance_sessions	session_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Session identifier
	offering_id	INT	—	NOT NULL	FK, INDEX	→ offerings
	component_id	INT	—	NOT NULL	FK	→ components

	session_date	DATE	—	NOT NULL	UK tuple	Held date
	timetable_entry_id	INT	—	NULL	FK	→ timetable_entries
	recorded_by	INT	—	NOT NULL	FK	→ technical_officers
attendance_records	attendance_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Attendance identifier
	session_id	INT	—	NOT NULL	FK	→ sessions, CASCADE if unposted removed
	enrollment_id	INT	—	NOT NULL	FK	→ enrolments
	is_present	BOOLEAN	—	NOT NULL	UK pair	Presence state

Table	Field	Data type	Size	Constraints	Key	Description
medical_records	medical_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Medical identifier
	student_id	INT	—	NOT NULL	FK, INDEX	→ undergraduates
	offering_id	INT	—	NULL	FK	Related offering where known
	medical_date	DATE	—	NOT NULL	—	Submission/evidence date
	start_date/end_date	DATE	—	NOT NULL	CHECK valid range	Excused period
	reason_details	VARCHAR	500	NOT NULL	—	Reason
	approval_status	ENUM	—	NOT NULL DEFAULT 'PENDING'	INDEX	PENDING/APPROVED/ REJECTED
	affects_eligibility	BOOLEAN	—	NOT NULL DEFAULT FALSE	—	Approved rule flag
course_materials	material_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Material identifier
	offering_id	INT	—	NOT NULL	FK	→ offerings
	lecturer_id	INT	—	NOT NULL	FK	→ lecturers
	title	VARCHAR	150	NOT NULL	—	Title
	file_path	VARCHAR	255	NOT NULL	—	Controlled resource path
notices	notice_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Notice identifier
	title	VARCHAR	180	NOT NULL	—	Heading
	body	TEXT	—	NOT NULL	—	Message
	audience	ENUM	—	NOT NULL	—	ALL/role/batch
	publish_date	DATETIME	—	NOT NULL	INDEX	Publish time
	expiry_date	DATETIME	—	NULL CHECK > publish	INDEX	Optional expiry
timetables	timetable_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Timetable identifier
	department_id	INT	—	NOT NULL	FK	→ departments
	semester_id	INT	—	NOT NULL	FK	→ semesters
	title	VARCHAR	100	NOT NULL	—	Timetable title
timetable_entries	timetable_entry_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Entry identifier
	timetable_id	INT	—	NOT NULL	FK, INDEX	→ timetables
	offering_id	INT	—	NOT NULL	FK	→ offerings
	component_id	INT	—	NOT NULL	FK	→ components
	day_of_week	ENUM	—	NOT NULL	INDEX	MON–SUN
	start_time/end_time	TIME	—	NOT NULL	CHECK start < end	Scheduled time
	location	VARCHAR	80	NOT NULL	—	Room/lab
grades	grade_id	INT	—	NOT NULL AUTO_INCREMENT	PK	Grade-scheme row
	scheme_version	VARCHAR	30	NOT NULL	UK tuple	e.g. UGC-12-2024

	grade_letter	VARCHAR	3	NOT NULL	UK tuple	Grade label
	min_mark/ max_mark	DECIMAL	5,2	NOT NULL	CHECK 0–100	Inclusive range
	grade_point	DECIMAL	3,2	NOT NULL CHECK 0–4	—	GPA value
audit_logs	audit_id	BIGINT	—	NOT NULL AUTO_INCREMENT	PK	Audit event
	user_id	BIGINT	—	NULL	FK, INDEX	Actor
	action	VARCHAR	50	NOT NULL	—	CREATE/UPDATE/etc.
	entity_name/ entity_id	VARCHAR/ BIGINT	80/ —	NOT NULL	—	Changed record
	occurred_at	DATETIME	—	NOT NULL	INDEX	Event time

19. Validation Rules

Field / input	Validation	Error message
Username	3–50 letters/digits/._; unique	Enter a unique username using 3–50 allowed characters.
Password	Minimum 8 characters; stored only as a hash	Password must contain at least 8 characters.
Student ID	Required, unique, permitted institutional format	Enter a unique valid student ID.
Email	Required valid email syntax; unique for account	Enter a valid email address.
Phone	Optional/required by profile policy; 7–20 permitted digits and + - spaces	Enter a valid phone number.
Course code	Required, unique, 3–20 uppercase letters/digits/hyphen	Enter a unique valid course code.
Credit value	Decimal greater than 0	Credit value must be greater than zero.
Assessment weight	0 < weight ≤ 100; total published plan = 100	Assessment weights must total 100%.
Marks	0 ≤ marks ≤ 100	Marks must be between 0 and 100.
Attendance	Computed 0% ≤ attendance ≤ 100%; one record per enrolled student/session	Attendance record is invalid or already exists.
Medical date/ range	Not future; end date not before start date; reason required	Enter a valid medical date range and reason.
Timetable	Start before end; no configured room/lecturer/course overlap	This timetable entry conflicts with an existing entry.

20. Error and Exception Handling

Error / exception	Java handling	User response
Invalid login	AuthenticationException; log attempt without password.	“Username or password is incorrect.”
Duplicate username/ enrolment	Pre-check; catch SQLIntegrityConstraintViolationException and map to ValidationException.	State the duplicate field/ relationship.
Database unavailable / SQL failure	DataAccessException wraps SQLException; transaction rollback; log technical cause.	Safe connection message; no SQL detail.
Invalid marks/attendance/ medical	Domain ValidationException before DAO call.	Show field-level correction message.
Missing required field	GUI and service validation.	Highlight required field.
Unauthorised operation	AuthorizationException, audit event.	“You do not have permission for this action.”
Record not found	NotFoundException from service/DAO result mapping.	“The requested record is unavailable.”

Use `try` for risk-prone operations, `catch` for specific exception types, `finally` or try-with-resources for cleanup, and custom exceptions for business-level errors. Controllers must not swallow errors; they log correlation details and return a recoverable GUI state.

21. GUI Requirements

All screens shall have a clear title, logged-in role/name, navigation, validation messages near inputs, confirmation before destructive/locking actions, and only role-authorised controls.

1. Login

Username/password fields, Login button, error label; all roles; validates active credentials.

2–5. Role Dashboards

Admin, Lecturer, Technical Officer, Undergraduate shortcuts, scoped alerts and summary counts; read-only summary data.

6. User Management

Admin: role selector, profile form, search table, Create/Update/Deactivate buttons; validates unique account identity.

7. Student Management

Admin: student/batch/contact/status fields and listing; validates student ID, batch and role link.

8. Course Management

Admin: course/offering/component/credit fields, assignment and enrolment tables; validates codes, credits, duplicates.

9. Attendance

Technical Officer: timetable session selector, component filter, presence grid, Save; lecturer/student view only by scope.

10. Medical Management

Technical Officer: student/course/date/reason/evidence/status controls; approval list and validation messages.

11. Marks Entry

Lecturer: offering/assessment/batch selectors, marks table, Save/Update; requires 0–100 and course assignment.

12. Eligibility

Lecturer: student/batch/course filters, attendance/CA/final status and reasons; student sees own status.

13–14. Grade / GPA Screens

Lecturer and student: courses, credits, final marks, grades, grade points, SGPA/CGPA; no manual edits.

15. Timetable

Admin editor; other role-specific read views; day/time/location grid with conflict validation.

16. Notice

Admin compose/publish/expire controls; all roles read targeted notices.

17. Course Materials

Lecturer add/edit/delete authorised materials; enrolled students view list.

18. Profile

Own profile fields; username/password locked; undergraduate contact/picture-only edit.

19. Reports

Authorised filters, preview table, print/export action; no raw data outside scope.

22. Reports

Report	Contents / filters	Authorised users
Student attendance	Student, course, component, present/scheduled/excused, percentage	Lecturer, Technical Officer, own student
Batch / course attendance	Batch/course/component summary and individual rows	Lecturer, Technical Officer
CA / final marks	Course, assessment, batch/student marks, weighted results	Lecturer; own student results
Eligibility	Attendance, CA, final status and reasons	Lecturer; own student
Grades / course results	Credits, final mark, grade, grade point	Lecturer; own student
SGPA / CGPA	Counted courses, credits, grade points and calculated values	Lecturer; own student
Medical records	Student, period, reason, approval and eligibility effect	Lecturer, Technical Officer, own student
Student profile	Permitted identity/contact/batch status	Admin, Lecturer; own student

23. Requirement Traceability Matrix

Requirement ID	Requirement	Module	Class	Database table	Use case
FR-001	Authenticate users	Authentication	User, AuthService	users, audit_logs	Login
FR-002	Maintain profiles	User Management	User + role subclasses	users, role tables	Manage Users
FR-003	Maintain courses/ timetable/notices	Course/Notice/ Timetable	Course, Timetable, Notice	courses, components, entries, notices	Manage Courses/Notices/ Timetables
FR-005	Manage materials	Course Materials	CourseMaterial	course_materials	Course Materials
FR-006/007	Configure and enter marks	Assessment/ Marks	Assessment, Marks, MarksService	assessments, marks	Upload Marks
FR-008/009	Theory/practical attendance	Attendance	AttendanceSession, AttendanceRecord	attendance_sessions, attendance_records	Manage Attendance
FR-010	Medical approval	Medical	MedicalRecord	medical_records	Medical Records
FR-011	Eligibility	Eligibility	EligibilityPolicy	enrolments, marks, attendance, medicals	View Eligibility
FR-012	Grades / SGPA / CGPA	Grade/GPA	Grade, GPA	grades, marks, enrollments	View Results/ GPA
FR-013	Notices/timetable	Notice/Timetable	Notice, TimetableEntry	notices, timetables, entries	View Notices/ Timetable
FR-014	Reports/audit	Reports	ReportService, AuditLog	all relevant tables, audit_logs	Reports

24. Assumptions and Limitations

24.1 Assumptions

Category	Assumption
Academic	Faculty supplies final approved course offerings, enrollment selection, assessment weights and any programme-specific policy exception.
Technical	All client computers can run the selected Java GUI runtime and reach the authorised MySQL server.
Database	Profile image files are stored in an access-controlled application directory and database stores only controlled paths/UUIDs.
Attendance	15 sessions are planned for each theory/practical component; approved medical absences are removed from the denominator (BR-009).
Grading	UGC Circular 12/2024 scheme applies to the relevant Technology-stream cohort; Faculty must confirm any local exception and repeat-result selection rule.
Security	Admin provisions initial accounts securely; password reset procedure is a controlled institutional process outside this mini-project scope.

24.2 Limitations

- Desktop application for a single institutional deployment; no mobile app or public Internet-facing portal.
- MySQL availability and local network connectivity are required for shared live data.
- Medical evidence is represented as validated details/reference; a public online upload portal is not included.
- The system calculates from approved configured data but does not replace formal examination-board approval.

25. Future Enhancements

- Web and mobile applications with secure single sign-on.
- Email and SMS notifications for notices, eligibility and timetable changes.
- Parent portal, online medical upload workflow, and document scanning.
- Advanced analytics for attendance/assessment trends and risk indicators.
- Cloud deployment, automated backups, and University LMS integration.

These items are explicitly future enhancements, not current functional requirements.

26. Conclusion

This SRS defines a feasible Java + MySQL GUI mini-project with scope-controlled roles, configurable academic assessment, correct theory/practical attendance treatment, medical approval, eligibility, grades, SGPA/CGPA, and an OOP-oriented layered design. The explicit assumptions protect the project from presenting unconfirmed local academic policies as official rules.

Appendix A — Sample Data

DB-003 Seed data shall include 1 Admin, at least 5 Lecturers, at least 4 Technical Officers, and at least 20 Undergraduates. The following example has 24 students and supports all required categories/scenarios.

Population	Example structure
Admin	ADM001 — one active administrator.
Lecturers	LEC001–LEC005 — five lecturers assigned across Faculty of Technology courses.
Technical Officers	TO001–TO004 — four officers, department scoped.
Normal undergraduates	STU001–STU016 — 16 active normal students in FOT-2025 batch.
Repeat undergraduates	STU017–STU020 — four course enrolments with student_status=REPEAT, attempt_no=2.
Batch-missed undergraduates	STU021–STU024 — four course enrolments with student_status=BATCH_MISSED in the appropriate later offering.

Attendance test case	Component example	Required demonstration
STU001	Theory 13/15; practical 14/15	>80% without medical
STU002	Theory 12/15; practical 12/15	Exactly 80%
STU003	Theory 10/15; practical 10/15	<80%, no medical
STU004	Theory 13/15 plus approved medical absence	>80%, with medical
STU005	Theory 8/15, approved medical does not bring percentage to 80%	<80%, with medical

Appendix B — Glossary

Term	Meaning
Academic year	Institutional year used with a semester to identify an offering.
Assessment plan	Configured assessment components and weights for one course offering.
Course offering	A course delivered to a selected batch in a selected semester/year.
Eligibility	Derived status based on the stated attendance and CA business rules.
Final mark	Weighted result derived from configured assessment marks.
Soft delete	Deactivating a record instead of destroying academic history.